PSEL4 compiles each H2 circuit four ways and keeps the cheapest

PSEL4 is a drop-in replacement for the default compile of pytket-quantinuum. It never costs more HQC than pytket level 2 or level 3 on a circuit where both pass its checks. Against level 2 (pytket's default) it saves 10.22 % HQC on 17 arithmetic and oracle circuits of 20 to 56 qubits whose output is proven equal to the input.

Most of that saving comes from choosing level 3 where it is cheaper. The ZX candidates add 0.26 % on top.

Team Learned ZX Rewriting

C.-F. Kam, A. Mahasinghe, D. De Silva, N. Karunarathne, F. Cadet, M. Bessafi, K. De Silva

Quantinuum Singapore Grand Challenge, Submission 1
October 2026

Blind ZX rewriting can raise the cost of an H2 run

A run on Quantinuum H2 is billed in HQC.

$$\mathrm{HQC} = 5 + C\left(N_{1q} + 10\,N_{2q} + 5\,N_{\mathrm{SPAM}}\right)/5000$$

C is the shot count. N_{1q} counts PhasedX gates. N_{2q} counts ZZPhase and ZZMax gates. N_{SPAM} counts initialisations, measurements and resets. Rz gates are free.

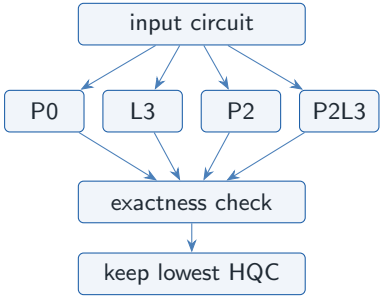
- ZX rewriting (PyZX teleport_reduce) placed before pytket's Quantinuum pass can lower this cost or raise it.
- Run blindly before level 2, it costs 6.27 % more HQC on 180 Trotter circuits and 22.1 % more on 3 QFT adders.
- On draper_adder_16 it costs 1,385.0 HQC against 906.8 for level 2.
- pytket's own `ZXGraphlikeOptimisation` followed by level 2 costs 59.22 % more than level 2 on 14 B1 circuits. Its documentation warns that it "may increase the cost of the circuit".
- Qiskit at optimisation level 3, rebased to H2, also costs more than level 2.
- A team with a fixed HQC budget cannot tell in advance which circuits gain. **[O]**

PSEL4 answers one question per circuit for a team on a fixed HQC budget

- **User.** A team that runs arithmetic and oracle subroutines on Quantinuum H2 from a fixed HQC budget. **[O]**
- **Decision.** Should this circuit go through ZX rewriting before pytket's Quantinuum pass, and at which optimisation level?
- **Answer.** PSEL4 compiles the circuit four ways and keeps the cheapest result that passes its checks. Level 2 and level 3 are always candidates.
- **Why per circuit.** On the 60 XXZ circuits teleport_reduce before level 2 raises the total HQC by 12.33 %. On other circuits it lowers the cost. The choice depends on the circuit.
- **What the user gains.** The user tries ZX rewriting on every circuit for the price of extra compilations and never pays more HQC than level 2 or level 3. **[O]**

PSEL4 keeps the lowest HQC of four pytket compilations

- **P0.** pytket-quantinuum 0.59.3 at optimisation level 2, pytket's default.
- **L3.** The same pass at optimisation level 3.
- **P2 and P2L3.** PyZX teleport_reduce, then level 2 or level 3.
- A candidate that fails its exactness check or times out is dropped. Ties go to the lower two-qubit count.
- **Guarantee.** Level 2 and level 3 are always candidates. PSEL4 therefore never costs more HQC than either on a circuit where both pass its checks. This holds by construction.



`pipelines/pass_select.py --psel4`

Assessment [O]. PSEL4 is pytket's PassSelector pattern with HQC as the metric. The contribution is the measured cost ledger, the guard and the correctness evidence. It is not a new selection mechanism.

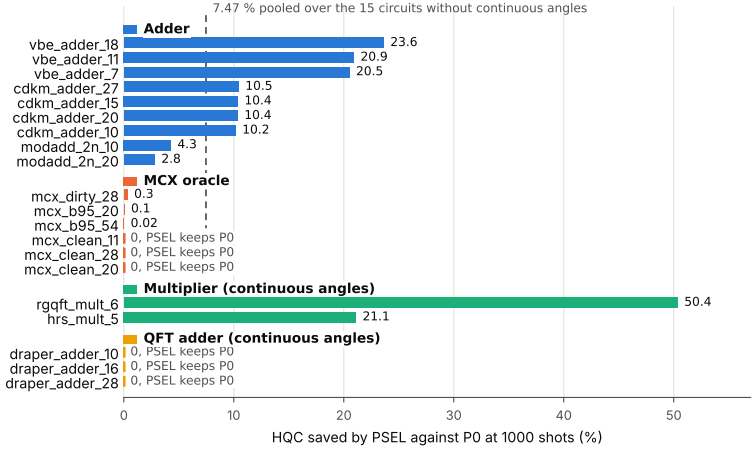
PSEL4 saves 10.22 % against level 2 on the 17 proven family C circuits

		HQC				PSEL4 saved %		
Set	Circuits	L2	L3	min(L2, L3)	PSEL4	vs L2	vs L3	ZX-only
Family C, proven output	**17**	11296.8	10224.2	10168.2	10142.2	**10.22**	0.80	0.26
Family C, all circuits	20	24535.0	18853.2	18797.2	18193.2	25.85	3.50	3.21
B1 Clifford+T benchmarks	15	15516.0	13965.8	13932.0	13863.4	10.65	0.73	0.49
A, Trotter circuits	180	16422.6	16231.2	15949.0	15949.0	2.88	1.74	0.00
Grover use case	2	2363.6	1582.0	1582.0	1575.0	33.36	0.44	0.44

HQC at 1000 shots. min(L2, L3) is the cheaper pytket level per circuit. ZX-only is PSEL4 against that minimum. Source `value/psel4_zx_only.md`.

- Most of the saving against level 2 comes from choosing level 3 where it is cheaper.
- The ZX candidates add 0.26 % on the 17 proven family C circuits, 0.49 % on B1, 0.44 % on the use case and 0 on the Trotter circuits.
- The 25.85 % on all 20 family C circuits includes 3 picks without a proof.

The earlier two-candidate selector gains most on adders and multipliers



Earlier two-candidate selector PSEL (P0 or P2) against level 2 (pytket's default). This figure does not show PSEL4. HQC at 1000 shots. A zero bar means PSEL keeps P0. Data `value/family_C_selector.csv`.

Level 3 is the strongest baseline and PSEL4 never costs more than it

	Total HQC					PSEL4 saved %	
Family	L2	L3	ZXG	QK3	PSEL4	vs L2	vs L3
B1 (14, hwb8 excluded)	2235.0	1928.6	3558.6	2799.2	1826.2	18.29	5.31
C (20)	24535.0	18853.2	53115.6	30707.2	18193.2	25.85	3.50

Total HQC at 1000 shots. ZXG is `ZXGraphlikeOptimisation` followed by level 2. QK3 is Qiskit at optimisation level 3 rebased to H2. hwb8 is excluded because its ZXG and P2L3 runs timed out.

- Level 3 costs 13.71 % less than level 2 on B1 and 23.16 % less on family C. It is the strongest pytket level.
- ZXG costs 59.22 % more than level 2 on B1 and 116.49 % more on family C.
- QK3 costs 25.24 % more than level 2 on B1 and 25.16 % more on family C.
- PSEL4 is never above min(L2, L3) on the 217 circuits where both levels passed. It is strictly below it on 14 of them.

Outputs are checked exactly up to 20 qubits and proven on 17 of 20 larger circuits

- **Exact checks.** PSEL4 checks every candidate up to 20 qubits. On families A, B1 and B2 every compiled circuit is checked against the uncompiled reference with tolerance 1e-8. All 1260 family A rows, 105 B1 rows and 100 B2 rows pass.
- **Miter proofs.** A PyZX miter reduction proves the PSEL4 output equal to its input as a full unitary up to global phase on 17 of the 20 family C circuits of 20 to 56 qubits.
- **Not proven.** hrs_mult_5 has off-grid angles and passes a check on 64 random basis inputs only. The miters of rgqft_mult_6 and draper_adder_28 timed out after 300 s.
- **Official pass.** Our level 2 pipelines give the same counts as the official pytket-quantinuum pass on all 445 compiled rows of families A, B1, B1x and B2. Family C was not compiled with the official pass.

A 22-qubit PSEL4 output returns the correct sum on the H2-2E emulator

Circuit	Input	Pipeline	Two-qubit gates	Shots	F_H	sd
vbe_adder_7, 22 qubits	a = 28, b = 111	P0	142	1000	0.883	0.0102
vbe_adder_7, 22 qubits	a = 28, b = 111	L3, PSEL4 pick	79	1000	0.901	0.0094
vbe_adder_3	all-zero	P0	56	4000	0.9520	0.0034
vbe_adder_3	all-zero	P2	41	4000	0.9628	0.0030

22 H2-2E jobs, 7202.4 HQC estimate. For vbe_adder_7 F_H is the share of shots with the correct sum 139.

- vbe_adder_7. The input was fixed and committed before submission. L3 is higher by 0.0180 with $z = 1.30$, not resolved at 1000 shots.
- vbe_adder_3, pooled over four runs. P2 is higher by 0.0108 with $z = +2.38$ and two-sided $p = 0.017$. The Holm-adjusted p is 0.035 over the two repeated circuits and 0.052 over all three H2-2E circuits.
- A repeats-only check gives a Holm-adjusted p of 0.030. It was chosen after the pooled result was known. grover_5 shows no gain. Both circuits ran on the all-zero input.

Assessment [O]. The vbe_adder_3 gain is not established at 0.05 after correction over three circuits.

A known T-count bound shows how much exact T optimisation can still gain

- Each T gate of a surface-code machine consumes one magic state. On H2 a T gate is a free Rz.
- On B1 the ZX stage removes 2,862 of 7,048 T gates of the written circuits, or 40.6 %.
- The floor is the known channel bound of Gosset et al. (2014), evaluated exactly in integer arithmetic. On 20 synthesised Haar circuits it is 0.997 of the T-count after ZX, pooled.
- Further exact T optimisation can gain at most 0.3 % pooled and 0.72 % on the worst instance.
- The evaluation covers 24 circuits of at most 5 qubits. It covers no circuit of family C.

Assessment [O]. The bound itself is not new. Its exact evaluation tells a fault-tolerant team where further T optimisation does not pay.

Limitations

- No user interview or survey is part of this submission.
- The ZX-only gain is small. On the 17 proven family C circuits it is 0.26 % on top of the cheaper pytket level.
- 3 family C picks are unproven. 87.6 % of the family C saving against level 3 comes from 2 of them, hrs_mult_5 and draper_adder_28.
- The vbe_adder_3 fidelity gain has a pooled Holm-adjusted p of 0.052 over three circuits. The repeats-only check gives 0.030 and was chosen after the pooled result.
- The earlier emulator runs of vbe_adder_3 and grover_5 used the all-zero input.
- HQC is the formula estimate. Nothing has run on H2 hardware.
- The T-count floor is the known Gosset et al. bound, evaluated on 24 circuits of at most 5 qubits.
- The selector follows pytket's PassSelector pattern.

Finals plan for H2 hardware and learned ZX rewriting

H2 hardware test

Test	Shots	Power	HQC
vbe_adder_3, recommended	4364	0.80	1147.3
vbe_adder_3, minimum	2900	0.65	765.7
grover_5, secondary	1900	0.65	2069.2

Shots per pipeline, P0 and P2.

- **[O]** A one-sided z of 1.645 or more confirms the gain. The rule is fixed before any hardware data.

Learned ZX rewriting

- **[O]** Learned ZX rewriting is the finals roadmap. A model will predict which ZX rewrites lower HQC. The four-candidate ledger rows are its training data.
- A first classifier on input-circuit features does not beat constant choices on held-out families. Logistic regression reaches a pooled balanced accuracy of 0.433 against 0.500 for the constant choices.
- **[O]** The finals model therefore needs structural features.

Every result can be rerun from the code package

- `code.zip`. The source of the repository. `README.md` gives the run instructions.
- `requirements.txt`. Pins 20 packages, among them pytket 2.18.4, pytket-quantinuum 0.59.3 and pyzx 0.10.7.
- `run_tests.sh`. Runs 28 test files, eight as scripts and twenty under pytest. The last run in `submission/build_logs/tests_summary.txt` shows rc=0 for all 28.
- `EVIDENCE.md`. Lists every emulator job with its job ID, shots, HQC estimate and stored payload.
- **Generated tables.** `report/make_tables.py` builds the results tables from `results/*.csv`. The value tables come from the scripts under `value/`. The circuit generators use seed 20261006.
- **Independent checks.** Verification reports by agents that did not produce the results are in `results/`.